

```

        Your name:
        <input size="40" type="text" id="name" />
        <input type="submit" />
    </form>
</div>
<script src="https://code.jquery.com/jquery-3.1.0.min.js">
</script>
<script type="text/javascript" src="/socket.io/socket.
io.js"></script>
<script>
    jQuery(function(){
        var socket = io.connect();
        var $nameForm = $('#name_form');
        var $name = $('#name');

        $nameForm.submit(function(e){

e.preventDefault();
        socket.emit('sent', $name.val());
        socket.val('');
        });

        socket.on('receive', function(data){
var html = '<h1>Welcome ' + data + '</h1><br /><i>This
message was generated using socket.io</i>';
$('#content').html(html);

        });

    });
</script>
</body>
</html>

```

When done, go to the terminal and at the command prompt, type the following command:

```
$ node test
```

Open your browser and type *localhost:3000*.

If you see the result shown in Figure 2, congrats! You did everything exactly right. Here, we linked a *socket.io* file in the script section or mark up, which provided us the client-side functionalities of *socket.io*. Later, using our form, we ask for an input from our user and when he submits the form, we trigger the *emit* function of the socket, which sends the filled-out data to our Express server, which then processes our data and sends it back to the client using another *emit* function. Each *emit* function is uniquely named, like 'send' and 'receive'.

 **Note:** All this should be done without refreshing the page! To ensure that, we use jQuery's *preventDefault* function when the form is submitted.

Now, you are ready to move to the most important part of this article.

Building a real-time application

We will now implement a LAN based chat engine, which you can use to communicate with all the people connected on the same network as you. But first, it's important to know how to access your apps on the local machine and access all the machines connected on the same network as your machine (your LAN).

Now, you can access your test app from any PC connected to your local network by just typing the IP address like this: "192.168.1.205:3000".

Just as in our previous test application, we need to create a *package.json* file and put the following code in it:

```

{
    "name": "test_chat_application",
    "version": "0.0.1",
    "private": "true",
    "dependencies": {
        "socket.io": "1.4.8",
        "express": "4.14.0"
    }
}

```

Let's run the script now by executing the following statement:

```
$ npm install
```

Next, let's create a file named *server.js*, which will work like *test.js* and use this code for your *server.js* file:

```

var express = require('express'),
app = express(),
server = require('http').createServer(app),
io = require('socket.io').listen(server),
nicknames = [];

server.listen(3000);

app.get('/', function(req, res){
    res.sendFile(__dirname + '/chat.html');
});

io.sockets.on('connection', function(socket){
    socket.on('new user', function(data, callback){
        if(nicknames.indexOf(data) != -1){
            callback(false);
        } else{
            socket.nickname = data;
            nicknames.push(socket.nickname);
            io.sockets.emit('usernames', nicknames);
            callback(true);
            updateNicknames();
        }
    });
});

```